

Build Your Own Coding Agent: Designing and Evaluating an Agentic Harness for Python Coding

Mohamed Ali El Euch

Applied AI Lab, University of Passau
Passau, Germany

Haseeb Ramzan

Applied AI Lab, University of Passau
Passau, Germany

Abstract

Large language models can propose code, but useful software-engineering agents also require a harness that controls what the model may do, grounds it in an unfamiliar repository, applies edits safely, executes feedback, and recovers from failure. We present BUILD YOUR OWN CODING AGENT, coding-agent harness for Python repositories, built around a six-state finite-state controller, a safety-checked file-system boundary, 33 selectively exposed tools, two-stage repository retrieval, task-local scratch memory, cross-task reflection, and trajectory-derived reliability guards. We evaluate the system in stages: HumanEval acts as a sanity check on the agent loop, while three controlled full runs on the 500-task SWE-bench Verified set measure repository-level behavior. The final Gemma 4 31B configuration resolves 185/500 issues (37.0%), compared with 161/500 (32.2%) for the same model before the guard set, while reducing recorded model tokens from 168M to 88M. Under the same final harness, Qwen3.6-35B-A3B resolves 173/500 (34.6%) but attains higher patch precision (55.3% for Qwen versus 45.9% for Gemma). A same-task configuration study further shows that retrieval initially increases exploration cost, whereas batch tools and guard logic reduce consumption from 768,825 to 174,488 tokens without changing correctness. Same-checkpoint release anchors place these results in context: our harness recovers a substantially larger fraction of Gemma’s externally demonstrated ceiling than of Qwen’s, exposing a harness–model fit gap. These results show that harness design affects both solution rate and computational efficiency; trajectory logs are therefore not merely debugging artifacts, but evidence for systematic agent improvement.

Keywords

coding agents, agent harness design, finite-state machine, tool use, retrieval-augmented generation, SWE-bench Verified, token efficiency, large language models

1 Introduction

Code generation becomes substantially harder when the target is not a single function but a real repository. A repository-level issue can require locating a behavior across hundreds of files, understanding project conventions, modifying an exact region without damaging nearby code, and validating the patch in an environment that may itself be incomplete. A large language model alone does not provide this operating discipline. The surrounding *agentic harness*; its controller, tools, context policy, memory, and recovery logic, determines whether model reasoning becomes a safe, testable patch or an expensive loop. This project asks how much useful software-engineering behavior can be obtained from locally served open models when the harness is built from first principles. Our design uses an OpenAI-compatible endpoint backed by vLLM [7],

but keeps model access behind a small client abstraction. The agent receives a natural-language task and a repository workspace, then iteratively plans, explores, edits, verifies, and returns a Git diff. Unlike an unrestricted chat loop, the harness exposes tools according to a finite-state machine (FSM), constrains file access to the workspace, and records every transition and tool outcome. We investigate three questions:

- **RQ1:** Can a compact, model-agnostic harness solve both function-level and repository-level Python tasks?
- **RQ2:** Which harness changes improve repository-level success and token efficiency under a fixed model and evaluator?
- **RQ3:** How does model choice change coverage, precision, and cost when the harness is held constant?

Our contributions are: (1) an end-to-end coding-agent architecture with a six-state controller, 33 tools, safe file access, two-stage retrieval, and two memory timescales; (2) a trajectory-driven debugging method that converts recurring failures into explicit guards; (3) a controlled three-run evaluation separating a harness ablation from a model ablation; and (4) a reproducible analysis of attempts, correctness, precision, and token use rather than reporting a single headline score.

2 Background and Related Work

2.1 Benchmarks for code generation and repository repair

HumanEval contains 164 hand-written Python function-completion tasks scored with hidden tests [1]. It exercises prompting, tool calling, editing, and execution, but each task is a single self-contained function, so it does not reproduce the search and environment challenges of maintenance work. SWE-bench instead derives tasks from real GitHub issues and repository states, requiring an agent to locate the relevant code in a large project and produce a patch that satisfies the repository’s own tests [6]. SWE-bench Verified is a human-validated 500-task subset designed to improve evaluation reliability [10]; later analysis nevertheless reported contamination and test-quality limitations [11], which we revisit in Section 8.3. The two benchmarks therefore measure different things, and we use both: the first as an inexpensive regression gate, the second as the controlled repository-level evaluation.

2.2 Agentic harnesses for software engineering

SWE-agent demonstrated that the interface between a model and a computer materially affects performance: purpose-built commands for viewing, editing, and navigating code raised resolve rates well above a naive shell loop [18]. Our work adopts that premise but studies it under local-model and course-project constraints, and

shifts the emphasis toward mechanisms absent from a plain agent-computer interface: a state-gated tool surface, a two-stage retrieval hierarchy, persistent trajectory evidence, and reliability guards derived from that evidence. Rather than reporting a single configuration, we treat the harness itself as the experimental variable and measure a cumulative guard set against its immediate predecessor under a fixed model and evaluator (Section 4.1).

2.3 Reasoning, retrieval, and reflective memory

ReAct, a prompting framework for tool-using agents, established the loop most such systems now follow: the model alternates between a reasoning step and a tool call, and each tool result is fed back as an observation that revises the next decision [19]. Our loop follows this pattern but makes the allowed action set state-dependent. Retrieval-augmented generation (RAG) grounds generation in external evidence [8]; repository retrieval is especially useful because source trees are larger than the context that should be sent on every turn. We use Sentence-BERT-style embeddings [14] and an embedded Chroma vector store [2]. Reflexion shows that verbal feedback can support improvement without weight updates [15]. Our implementation separates volatile per-task notes from short cross-task reflections stored on disk.

3 System Design

3.1 Task contract and interfaces

The core contract is intentionally small: the harness receives a repository path, task description, model configuration, and resource limits; it returns a candidate patch, execution record, and trajectory. The same core loop supports a command-line interface and a FastAPI/Vite browser interface with a terminal-like interaction surface. Decoupling interfaces from the agent keeps evaluation non-interactive while preserving a demonstrable development environment (Appendix B). Figure 1 shows how the pieces fit together. Five layers sit between a task and the repository: the interfaces that accept the task, the FSM controller that decides which tools are legal at each step (Section 3.2), the reasoning layer that proposes the next action, the capability layer of 33 typed tools (Section 3.4), and a workspace boundary through which every file operation passes (Section 3.3). Three subsystems feed that loop rather than sitting inside it—two-stage retrieval (Section 3.5), scratchpad and reflection memory (Section 3.6), and the reliability guards (Section 3.7). The model proposes; the controller, tool layer, and boundary determine what can actually change on disk. Each remaining subsection opens one layer.

3.2 Finite-state control

The controller represents progress with six states: PLAN, EXPLORE, IMPLEMENT, VERIFY, FIX, and DONE. The model can request a transition, but the controller validates it and changes the tools exposed on the following turn. For example, early exploration prioritizes listing, outlines, search, and reading; implementation enables mutation; verification exposes execution and Git inspection. This mechanism serves two purposes: it reduces irrelevant schemas in context and prevents an early speculative edit from becoming the default behavior. Figure 2 shows the main path and recovery edges. Transitions are event-driven in addition to model-requested: a passing test

run moves the loop to DONE from any state that exposes test execution, a failing run moves it to FIX, and a completed edit forces re-verification; new evidence can return the agent to EXPLORE. The final patch is extracted as a Git diff after termination. Iteration and token limits provide a hard outer termination condition. The entry state itself is selected by a workspace inspection at task start: large unfamiliar codebases begin in PLAN, small workspaces in EXPLORE, and an existing partially written solution file can begin directly in IMPLEMENT or VERIFY. On SWE-bench this always resolves to PLAN; on HumanEval it saves two to three iterations per task.

3.3 Safe file-system boundary

All file-oriented tools call a shared `FileSystem` class. Its `_resolve_path` routine converts a requested path to an absolute path and rejects anything outside the workspace root, blocking `../` traversal. Repository enumeration uses `git ls-files`, so agent search operates on tracked source and respects ignore rules rather than walking caches, build artifacts, virtual environments, or Git internals. Centralizing this policy gives reading, search, and editing the same security semantics. Edits are observable and reversible. Exact replacement refuses ambiguous matches; combined edit-and-verify operations perform a syntax check and return a diff; Git tools expose status, diff, history, commit, and single-file restoration. Shell execution remains a powerful capability, so the harness gates commands against a prefix allowlist (e.g., `python`, `pip`, `pytest`, `git`), restricts the working directory, and records commands and results in the trajectory.

3.4 Capability layer: 33 tools

The final registry contains 33 tools grouped by intent: navigation (4), reading (5), search (3), editing and file mutation (8), execution and testing (4), Git (5), and memory/control (4). Some tools combine operations that previously required several turns. `search_and_read` retrieves and opens top hits; `edit_and_verify` applies an edit, checks syntax, and reports the change; `edit_files` batches independent edits. Appendix A lists the full surface. Selective exposure is as important as tool count. The controller sends only schemas relevant to the current state, reducing prompt overhead and making the next legal action more obvious. Batch tools then reduce the number of model round trips once the correct action is known.

3.5 Two-stage repository retrieval

At task start, tracked Python files are parsed into an abstract syntax tree (AST) with the Python standard library’s `ast` module [12]. Functions and classes become coherent chunks with file, symbol, and line metadata; the path, the name of the enclosing function or class, and the start and end line numbers of the unit in its source file. Files that do not parse fall back to fixed-size chunks. Structural chunking matters because retrieval quality in code search depends on returning units that respect syntactic boundaries rather than arbitrary character windows [5]. An `all-MiniLM-L6-v2` sentence-transformer [14]—a six-layer distilled MiniLM encoder [17] producing 384-dimensional embeddings—embeds both code chunks and compact file summaries. The implementation creates a fresh in-memory client per task in Chroma [2], an embedded vector database that stores embeddings alongside their metadata and answers

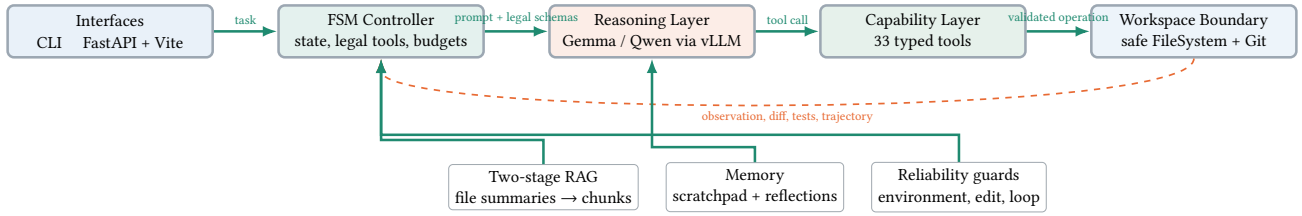


Figure 1: Five-layer harness architecture. Retrieval, memory, and reliability guards feed the decision loop rather than sitting inside it.

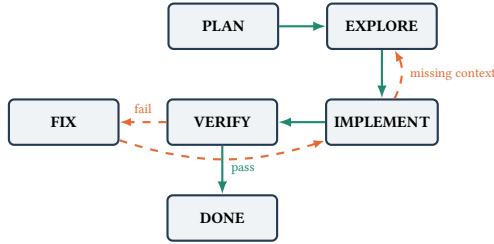


Figure 2: Six-state workflow (canonical path). Dashed edges represent recovery; in the implementation transitions are event-driven, and a passing test run reaches DONE from any state exposing test execution.

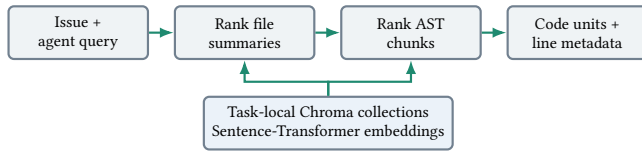


Figure 3: Two-stage RAG narrows by file before returning structural code chunks.

nearest-neighbour queries, with separate summary and chunk collections; it does not claim persistence across tasks. Retrieval is hierarchical (Figure 3). A query first ranks file summaries, narrowing the repository. The second search ranks code chunks across the index and promotes chunks originating from the selected files, so file-level evidence re-orders the chunk-level results. This follows the retrieve-then-rerank pattern used in multi-stage ranking systems [9, 16], in a deliberately lightweight form: the promotion is a rule applied to first-stage evidence rather than a learned scoring model. Coarse-to-fine retrieval of this kind is a standard response to the precision loss that occurs when a single flat search is run over many thousands of short passages [8]. Exact text and regular-expression search remain available for known identifiers; semantic search is used when issue vocabulary and implementation vocabulary differ.

3.6 Memory and context management

Three mechanisms operate at different timescales, and separating them prevents the common failure in which a single growing message list has to serve as plan, evidence, and history at once.

Within a turn: context trimming. The conversation grows by one assistant message and one tool result per iteration, and tool results are the largest contributors; a file read or a test log can exceed the rest of the exchange. When the accumulated history approaches the model’s usable context, the oldest low-value observations are dropped first, recent interaction is preserved, and the leading system content is never removed. Trimming is therefore lossy by design; the next two mechanisms exist because of it.

Within a task: the scratchpad. The agent writes its own notes through a tool, `write_scratchpad(key, value)`, storing whatever it judges worth keeping; the current hypothesis, the file and line where the defect appears, what has already been ruled out, the state of verification. Notes are held in a per-task dictionary and serialized before every model call into a labelled section that is re-injected as system content at a fixed position, ahead of the conversation proper. Because injection is repeated on each iteration, the notes cannot be trimmed away: the agent may lose the turn in which it made a discovery, but not the discovery. The dictionary is cleared at the start of each task, so nothing leaks between instances. In one recorded trajectory the scratchpad held a correct 2,000-character root-cause analysis while successive edit attempts were still failing, and only the application had to be repaired.

Across tasks: reflection. After a failed task, a short record is written to disk describing what was attempted and what should be done differently. At the start of a subsequent task the three most recent records are loaded and injected as system content in the same way. Records are chosen by recent timestamp alone; there is no similarity matching against the current task, so this is a lightweight form of experience replay rather than semantic memory, and we do not claim it retrieves *relevant* lessons. A representative record reads: avoid spending iterations on dependency installation; verify the patch with `git_diff` instead. Reflection is a natural place for similarity-based retrieval, which we leave to future work (Section 8.4).

Independently of all three, a resource guard caps recorded tokens per task and terminates the loop when the budget is exhausted, and every tool call, tool result, state transition, token count, and terminal reason is appended to a trajectory log. The trajectory is not part of the agent’s context: it is written for offline analysis, and it is the evidence base for the guards in Section 3.7 and the failure analysis in Section 7.

3.7 Reliability guards derived from trajectories

Three recurring failure modes motivated the final A4 guard set:

- (1) **Broken test environment.** Some historical repositories cannot collect tests in the generation environment because of dependency or configuration errors. The harness recognizes dependency-error patterns (e.g., `ModuleNotFoundError`, `ImportError`, “could not determine”, “broken installation”). After two detections it stops recommending futile reruns and asks the agent to validate with source inspection and `git diff`; authoritative correctness still comes from the later Docker evaluator.
- (2) **Stale or inexact edits.** Exact replacement fails if the model invents whitespace or edits from stale context. The prompt now requires a fresh read immediately before mutation, exact copying of `old_str`, and re-reading after a failed match. Editing tools also return closest-match guidance where possible.
- (3) **Repetition and analysis paralysis.** If the last three normalized tool results are identical, the controller injects a different approach nudge.

These mechanisms do not solve the issue for the model. They convert recognizable non-progress into a bounded, visible change of strategy.

4 Development Methodology

4.1 Configuration stages

Harness features were not added all at once, and reporting only the final system would hide which additions mattered. We therefore name four cumulative configurations (Table 1), each a superset of its predecessor, so that every performance claim maps to a concrete set of mechanisms.

A1 is the minimal action loop: read a file, write a file, list a directory, run tests. It is sufficient for single-file tasks, but in a large repository it can locate code only by reading files one at a time.

A2 adds the navigation and retrieval layer of Section 3.5; two-stage semantic search, file outlines, single-function extraction, and line-range views. This is the change that makes repository-level work possible at all.

A3 adds the batch tools which are `read_files`, `edit_files`, and `search_and_read`, each replacing a sequence of round trips with a single call, together with the surgical editing operations of Section 3.3.

A4 adds the three reliability guards of Section 3.7. It is the final configuration used in two of the three controlled runs defined in Section 5.3.

The stages are cumulative rather than factorial. That is a deliberate trade: it mirrors how the system was actually built and keeps the number of 500-task runs affordable, but it means the A3–A4 comparison estimates the guard set as a package rather than isolating individual guards (Section 7.2). A1 and A2 appear only in the same-task cost study of Section 6.3; neither was run over the full benchmark.

4.2 Trajectory-driven iteration

Development followed a repeated evidence loop: run a small task set, inspect empty or incorrect trajectories, group failures by mechanism, implement the smallest guard or tool change, then rerun before spending the full benchmark budget. HumanEval provided cheap regressions. A ten-task Astropy sample exposed repository

Table 1: Cumulative harness configurations.

Config	Core loop	RAG/nav.	Batch	Guards
A1	✓	–	–	–
A2	✓	✓	–	–
A3	✓	✓	✓	–
A4	✓	✓	✓	✓

navigation and historical-environment failures. On this sample, the successive guard introductions reduced empty patches from four to two to zero before any configuration advanced to a full run. Only stable configurations advanced to a 500-task run. This process is summarized as *observe* → *classify* → *intervene* → *measure*; it treats agent behavior as an experimental object rather than relying on prompt intuition alone.

5 Experimental Methodology

5.1 Benchmarks and metrics

HumanEval pass@1, the fraction of tasks solved by a single sampled attempt [1], is here the fraction of the 164 tasks solved by the emitted implementation under hidden tests. For SWE-bench Verified, a task is resolved only when the official evaluator applies the generated patch in the task’s Docker image and both test sets defined by the benchmark are satisfied [6]: the fail-to-pass tests, which the issue’s reference fix is expected to repair, must pass, and the pass-to-pass tests, which guard against regressions elsewhere, must remain green. We report four complementary quantities. *Resolved rate* is resolved tasks divided by 500 and is the primary correctness metric. *Patch rate* is non-empty patches divided by 500 and measures coverage, not correctness. *Patch precision* is resolved tasks divided by non-empty patches. *Recorded tokens* are aggregate model tokens logged by our harness; they support comparisons within our setup but are not normalized wall-clock cost. Of these, only the resolved rate is standard in the SWE-bench literature; patch rate, patch precision, and recorded tokens are diagnostic quantities we define here to separate coverage from correctness and to make cost comparable across configurations.

5.2 Models

Both models were served on the university InnKube infrastructure through the same vLLM-backed OpenAI-compatible client, with decoding parameters left at the endpoint defaults in both cases, so no sampling advantage is introduced on either side. The pair was chosen as an architectural contrast rather than for convenience. Gemma 4 31B is a dense, general instruction-tuned model in which all parameters are active for every token, with a 256K context window according to its model card [4]. Qwen3.6-35B-A3B is a mixture-of-experts model: of its 35B total parameters roughly 3B are active per token, it is trained for agentic use with interleaved reasoning, and it has a native 262,144-token context window [13]. These specifications describe the checkpoints, not the amount of context actually consumed by each task; in practice the per-task token cap below binds long before either context limit does.

Table 2: Controlled full-run design. All runs use the same 500 tasks, evaluator, 30-iteration limit, and 200,000 per-task token cap.

Run	Model	Config.	Controlled purpose
1	Gemma 4 31B	A3	pre-guard baseline
2	Gemma 4 31B	A4	harness ablation
3	Qwen3.6-35B-A3B	A4	model ablation

5.3 The three controlled runs

Table 2 defines the grid. Each run executes all 500 SWE-bench Verified tasks under identical budgets and is scored by the same official evaluator; the runs differ in exactly one factor at a time.

Run 1 pairs Gemma with configuration A3; the system before the reliability guards, and serves as the baseline. **Run 2** pairs the same model with A4, so the only difference from Run 1 is the guard set. **Run 3** pairs Qwen with A4, so the only difference from Run 2 is the model.

The design therefore supports two clean comparisons. Run 1 versus Run 2 is a *harness ablation*: model, tasks, budgets, and evaluator are fixed, and any difference is attributable to the guards (**RQ2**). Run 2 versus Run 3 is a *model ablation*: harness, tasks, budgets, and evaluator are fixed, and any difference is attributable to the check-point (**RQ3**). What the grid does not support is an interaction test: a fourth run pairing Qwen with A3 would be required to determine whether the guard set helps both models equally, and we did not have the compute budget for it. Section 7.2 returns to this limit.

Budgets and operational settings. Each task is capped at 30 agent iterations and 200,000 recorded tokens; reaching either terminates the loop and whatever diff exists at that point becomes the prediction. Transient endpoint failures (timeouts, rate limits, server errors) receive up to three total attempts with linearly increasing delays of 5, 10, and 15 seconds, necessary in practice, since Run 2 and Run 3 partly executed in parallel against the same endpoint. Predictions are written after every task and completed instance identifiers are skipped on restart, so a multi-day run survives interruption. Finally, generation and evaluation are deliberately separated: the generation host cannot reconstruct each repository’s historical dependencies, whereas the official Docker evaluator can, and only the latter determines whether a task counts as resolved.

6 Results

6.1 Function-level regression check

The final loop solves 158/164 HumanEval tasks, or 96.3% pass@1, with a recorded average of 2.0 iterations per task. This result shows that basic planning, editing, and execution usually function on isolated Python problems. It does not answer the repository-level research questions: the remaining analysis therefore uses SWE-bench Verified.

6.2 Repository-level correctness, coverage, and cost

Table 3 and Figure 4 report the three full runs. Run 2 is the strongest overall: 185 resolved issues (37.0%), 403 patches, and 88M tokens.

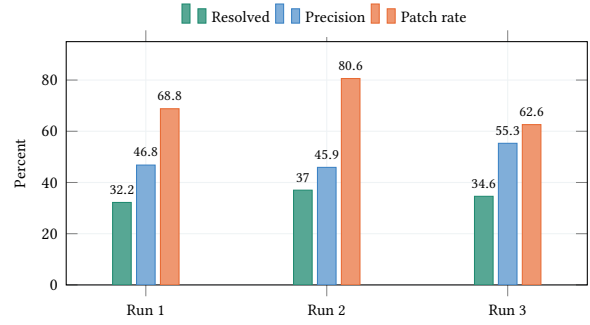


Figure 4: Correctness, precision, and coverage expose different behavior. A high patch rate alone is not evidence of correctness.

Run 1 resolves 161 issues (32.2%) from 344 patches with 168M tokens. Run 3 resolves 173 issues (34.6%) from 313 patches with 115M tokens.

Answer to RQ2: harness ablation. Run 1 and Run 2 change the harness while holding the model, tasks, budgets, and evaluator fixed. A4 produces 59 additional patches and resolves 24 additional issues, a +4.8 percentage-point gain. At the same time, aggregate tokens fall by 80M (47.6%). Token efficiency rises from 0.96 to 2.10 resolved issues per million tokens. Patch precision is essentially unchanged (46.8% to 45.9%); the improvement comes from converting stalled trajectories into more attempts without lowering the number of correct outcomes. The gain is also not uniform across repositories: the Django resolve rate dips slightly (59% to 56%), a visible cost of forcing decisiveness, while the mid-difficulty group (SymPy, Sphinx, scikit-learn, Matplotlib) rises from 28% to 35%, precisely the repositories where Run 1 most often produced empty patches. This supports the claim that environment, edit, and loop guards improve both coverage and efficiency.

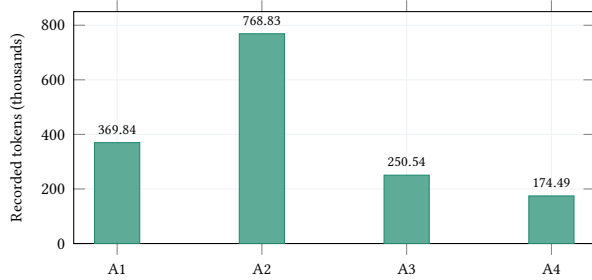
Answer to RQ3: model ablation. Run 2 and Run 3 hold A4 and evaluation fixed. Gemma creates 90 more patches, resolves 12 more tasks, and consumes 27M fewer tokens. Qwen produces fewer patches but has substantially higher precision: 55.3% versus 45.9%. A HumanEval sample illustrates the cost mechanism: on identical tasks that both models solve, Qwen averages 2.8 iterations and roughly 3.4× the tokens of Gemma, because interleaved reasoning traces are emitted, and therefore recorded, as ordinary tokens. Thus Gemma is the stronger default under this budget, while Qwen is more selective. The difference suggests that model-aware routing or reasoning-format support may outperform a single uniform policy, but our data do not yet test such a router.

6.3 Token economy across configuration stages

To isolate the cost of tool-surface changes, we replayed the same Astropy issue across A1–A4; all four configurations produced the same correct fix. Figure 5 shows a non-monotonic result. Adding retrieval in A2 increases consumption from 369,840 to 768,825 tokens because the agent can explore more broadly. A3 batch tools then reduce cost to 250,535 by combining search/read and edit/verify operations. A4 guards reduce it further to 174,488. Relative to

Table 3: SWE-bench Verified results from the official Docker evaluator. Precision is "resolved / patches"; token efficiency is resolved issues per million recorded tokens.

Run	Model / config.	Patches	Patch rate	Tokens	Resolved	Resolve rate	Precision
1	Gemma 4 31B / A3	344	68.8%	168M	161	32.2%	46.8%
2	Gemma 4 31B / A4	403	80.6%	88M	185	37.0%	45.9%
3	Qwen3.6-35B-A3B / A4	313	62.6%	115M	173	34.6%	55.3%

**Figure 5: Same correct Astropy patch under four cumulative configurations. Retrieval expands useful exploration before batch operations and guards reduce round trips and non-progress.**

the A2 peak, the final configuration (A4) uses 77.3% fewer tokens; relative to A3 it uses 30.4% fewer. The comparison is a one-task mechanism study, not an estimate of average benchmark savings, but it explains why capability and efficiency did not improve at the same stage.

7 Failure Analysis

7.1 Empty patches as diagnostic evidence

Run 1 produced 156 empty patches. An empty prediction necessarily fails, but its trajectory identifies where the workflow stopped: before localization, during an edit mismatch, after repeated non-starting tests, or at the resource limit. The A4 guard set targets the three recurring mechanisms described in Section 3.7. The increase from 344 to 403 non-empty patches is consistent with those interventions, although the aggregate evaluation cannot assign each additional patch to one guard independently. Run 3 adds a repository-level anomaly: Qwen produced no patch on any of the 41 Sphinx tasks. Two hypotheses are consistent with the aggregate data. A behavioral explanation is that the model remained in exploration on this documentation-tooling domain until reaching its limits. An infrastructural explanation is that these tasks executed while two benchmark runs shared the serving endpoint and transient timeouts peaked, so exhausted retries terminated tasks early with no patch. The per-task records can tell the two apart. If those tasks consumed tokens close to the cap, the agent was exploring until it ran out; if they consumed very little, they were terminated early. We plan this trajectory analysis as follow-up work. The remaining no-patch cases could be hidden rather than fixed. Many systems emit a best-effort diff: whatever the agent changed in the workspace is submitted at the end of the run, even

when the agent never confirmed the change was relevant. Since an empty prediction fails automatically, submitting something costs nothing and can only help, so patch rates near 100% are straightforward to obtain. We do not do this. Emitting arbitrary changes can improve patch rate cosmetically without improving resolve rate. We therefore retain patch rate as a diagnostic measure and treat resolved rate as the outcome. A structural factor amplifies all three mechanisms: generation runs without the repositories' executable historical environments (the generation host provides no Docker), so the agent never receives a failing-test signal to localize by or iterate against. Systems that generate inside the task's evaluation image obtain that feedback at every step; closing this gap is the highest-leverage single improvement available to this harness, and we place it first in future work (Section 8.4).

7.2 What the guards do and do not establish

The comparison between the last two configurations (A3–A4) measures the guard set as a package. Isolating the three guards individually would require a separate full run for each, which the compute budget did not allow, so we cannot say how much of the gain belongs to broken-environment detection, how much to the editing protocol, and how much to loop detection. Prompt text and tool behavior changed together inside A4, so causal credit cannot be split precisely among broken-environment detection, strict editing, and loop nudges. The trajectory examples supply mechanism evidence, while the full run supplies system-level outcome evidence. A future factorial experiment should enable each guard independently on the same fixed task sample. HumanEval failures also caution against treating repository context as the only difficulty. The six unsolved function tasks can arise from incorrect algorithm selection, edge cases, or termination before a correct implementation. Conversely, a high HumanEval result does not guarantee localization, dependency handling, or cross-file consistency. This gap explains why the 96.3% sanity check coexists with a 37.0% repository-level result.

8 Discussion

8.1 Answers to the research questions

For **RQ1**, the same harness spans function completion and real repository issues: it reaches 96.3% on HumanEval and 37.0% on SWE-bench Verified with the strongest tested run. The difference quantifies the added difficulty of repository search, environment reconstruction, and multi-step modification. For **RQ2**, the controlled A3–A4 comparison shows that explicit non-progress handling is not merely defensive. Under the same Gemma model, A4 improves resolved rate by 4.8 points and nearly halves recorded tokens. The same-task study adds an architectural explanation: RAG increases

Table 4: SWE-bench Verified under identical checkpoints. Reference scores come from the Qwen3.6 release’s internal scaffold (bash + file-editing tools, temperature 1.0; turn and token budgets unspecified) [13]. Our rows use the A4 harness. Bold marks the two checkpoints evaluated in both settings.

Model	Reference	This work
Qwen3.5-27B	75.0%	–
Qwen3.5-397B-A17B	76.2%	–
Gemma 4 31B	52.0%	37.0%
Claude 4.5 Opus	80.9%	–
Qwen3.6-35B-A3B	73.4%	34.6%
Qwen3.6-27B	77.2%	–

reachable evidence, batch tools reduce interaction overhead, and guards suppress repeated work. A harness must therefore be evaluated as both a capability system and a resource-allocation policy. For **RQ3**, a model cannot be summarized by one score independent of the harness. Under identical A4 settings, Gemma has higher coverage, higher resolved rate, and lower aggregate token use, while Qwen has higher conditional precision. This is actionable: a future system can route tasks based on repository, early localization confidence, or estimated reasoning needs, and can adapt budgets to model-specific behavior.

8.2 External context under identical checkpoints

No external system shares our harness, so absolute cross-system rankings are not meaningful. The Qwen3.6 release, however, reports SWE-bench Verified scores for several models; including both checkpoints evaluated in this work, measured under a single internal agent scaffold based on bash and file-editing tools at temperature 1.0 [13]. Table 4 reproduces those figures next to ours. Because the model checkpoints and the official evaluator are identical, the remaining difference isolates the surrounding system: scaffold behavior, generation environment, and budgets. Two observations follow. First, our harness recovers roughly 71% of Gemma’s externally demonstrated ceiling (37.0/52.0) but only about 47% of Qwen’s (34.6/73.4). The asymmetry indicates a *harness–model fit gap* rather than uniform harness quality: the same controller and tools exploit the dense generalist far better than the agentially trained reasoning model. This reading is consistent with the internal evidence; the Sphinx anomaly, Qwen’s reasoning-token overhead, and its stricter serving-format requirements. Second, the reference landscape itself is informative: agentially trained 2026 models cluster between 73% and 81%, while the general-purpose dense Gemma reaches 52%, so a substantial share of recent progress is attributable to agent-oriented training rather than scale alone. For era context, the GPT-4-based SWE-agent baseline reported roughly 23% on Verified [10] and the early autonomous system Devin about 13.9% on a SWE-bench subset [3]; both lie below our weakest run. The comparison remains bounded. The reference scaffold’s turn and token budgets are unspecified, its generation setting very plausibly includes executable task environments (Section 7.1), and newer checkpoints carry contamination risk on absolute values [11]. We

therefore treat the reference figures as upper bounds that motivate the budget-matched, Qwen-aware rerun listed in future work, not as an ablation.

8.3 Threats to validity and limitations

Benchmark validity. SWE-Verified bench has known contamination and test-quality concerns at current frontier performance [11]. We use it for controlled internal comparisons, not as evidence of a frontier rank. HumanEval is also small and contamination-prone.

Measurement scope. Token counts come from our serving and logging path and do not include every systems cost (index construction, Docker evaluation, waiting, or hardware energy). Aggregate tokens can also be affected by endpoint formatting. We therefore avoid price claims and compare only within the experiment.

Ablation granularity. Run 1 versus Run 2 isolates A3 from A4 but not each guard. The A1–A4 cost chart is a single successful issue and demonstrates mechanism rather than population-level effect. Repeated trials would be required to quantify sampling variance.

Model generality. Only two locally available checkpoints were evaluated. Generation parameters, prompt compatibility, reasoning-token behavior, and serving implementation may favor one model. Results should not be generalized to all Gemma or Qwen variants.

External anchors. The same-checkpoint comparison in Section 8.2 controls the model and evaluator but not the scaffold’s budgets or generation environment; the derived ceiling-recovery ratios are therefore indicative rather than exact.

Retrieval and memory. The current vector index is recreated for each task, which adds cold-start work. Cross-task reflections are selected by recency rather than similarity and were not independently ablated. The shell tool also remains a broad capability whose safety depends on deployment isolation in addition to path validation.

8.4 Future work

The immediate priorities are: (1) generating inside each task’s evaluation image, so that the agent can reproduce the failing test, localize from a real traceback, and iterate against genuine execution feedback rather than working blind; (2) a budget-matched rerun with Qwen-specific reasoning and system-message handling; (3) per-guard and per-tool ablations with repeated seeds; (4) a model router that predicts coverage-versus-precision trade-offs from early trajectories; (5) persistent, incrementally updated repository indexes; (6) similarity-based reflection retrieval with measured benefit; (7) best-of- k evaluation reported separately from single-attempt results; (8) evaluation on a newer, contamination-resistant benchmark while retaining the present suite for regression continuity; and (9) trajectory-level analysis of the Qwen–Sphinx anomaly.

9 Individual Contributions

Mohamed Ali El Euch. Mohamed led the core harness architecture and was responsible for the SWE-bench environment and evaluation setup, execution and management of the benchmark runs, and analysis of their trajectories and official Docker results. He developed major system components, including FSM-controlled operation, repository navigation and retrieval, the tool and file-system layers, and scratchpad, reflection, and trajectory infrastructure. He

also contributed to system integration, testing, documentation, and preparation of the report and presentation.

Haseeb Ramzan. Haseeb was responsible for the initial project scaffold and basic workflow, the user-interface layer, and dependency and requirements integration. He developed the HumanEval benchmark integration and agent prompts, implemented agent-history and tool-call context handling, and contributed targeted reliability optimization through improvements to editing and termination behavior. He also contributed to system testing, evaluation analysis, documentation, and preparation of the report and presentation.

10 Conclusion

BUILD YOUR OWN CODING AGENT demonstrates that a coding agent is more than a model wrapped in a loop. State control, a coherent

tool interface, safe repository access, retrieval, trimming-resistant memory, and explicit recovery behavior determine whether reasoning reaches an evaluable patch. On SWE-bench Verified, the final guard-equipped Gemma harness raises resolved issues from 161 to 185 while cutting recorded tokens from 168M to 88M. Holding the harness fixed reveals a different Qwen trade-off: lower coverage but higher patch precision, and same-checkpoint release anchors show that the remaining headroom is concentrated on the reasoning model; a harness-model fit gap rather than a uniform capability ceiling. The central result is therefore methodological as well as numerical: controlled runs and trajectory evidence make harness improvements explainable. The next step is not simply to add more tools, but to measure which controls help which models on which tasks.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [2] Chroma. 2026. Chroma Documentation. <https://docs.trychroma.com/> Accessed 15 July 2026.
- [3] Cognition Labs. 2024. SWE-bench Technical Report. <https://cognition.ai/blog/swe-bench-technical-report> Accessed 15 July 2026.
- [4] Google DeepMind. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core_model_card_4 Accessed 15 July 2026.
- [5] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *arXiv:1909.09436* <https://arxiv.org/abs/1909.09436>
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626. doi:10.1145/3600006.3613165
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [9] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv:1901.04085* <https://arxiv.org/abs/1901.04085>
- [10] OpenAI. 2024. Introducing SWE-bench Verified. OpenAI Research. <https://openai.com/index/introducing-swe-bench-verified/> Accessed 15 July 2026.
- [11] OpenAI. 2026. Why We No Longer Evaluate SWE-bench Verified. OpenAI Research. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/> Accessed 15 July 2026.
- [12] Python Software Foundation. 2026. ast — Abstract Syntax Trees. Python Standard Library Documentation. <https://docs.python.org/3/library/ast.html> Accessed 15 July 2026.
- [13] Qwen Team. 2026. Qwen3.6-35B-A3B Model Card. <https://huggingface.co/Qwen/Qwen3.6-35B-A3B> Accessed 15 July 2026.
- [14] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. 3982–3992. doi:10.18653/v1/D19-1410
- [15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652. <https://arxiv.org/abs/2303.11366>
- [16] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A Cascade Ranking Model for Efficient Ranked Retrieval. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 105–114. doi:10.1145/2009916.2009934
- [17] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33. 5776–5788. <https://arxiv.org/abs/2002.10957>
- [18] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. <https://arxiv.org/abs/2405.15793>
- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

A Complete Tool Surface

The final source registry contains the following 33 tools, grouped here by primary role:

- **Navigation:** `list_files`, `view_directory`, `find_files`, `explore_repo`.
- **Reading:** `read_file`, `read_files`, `view_file_range`, `file_outline`, `get_function`.
- **Search:** `search_code`, `search_codebase`, `search_and_read`.
- **Editing:** `edit_and_verify`, `edit_files`, `search_and_replace_all`, `write_file`, `str_replace`, `insert_at_line`, `delete_lines`, `create_directory`.
- **Execution:** `run_code`, `run_tests`, `run_command`, `generate_test`.
- **Git:** `git_status`, `git_diff`, `git_commit`, `git_log`, `git_checkout_file`.
- **Control:** `write_scratchpad`, `read_scratchpad`, `request_transition`, `report_confidence`.

Some tools span roles; the grouping is explanatory rather than an access policy. Authoritative registration is the schema list passed to the agent, and the FSM exposes a state-specific subset rather than all schemas on every turn.

B Interaction Surfaces

Figure 6 shows the two interactive front-ends built on the same agent core: the terminal chat interface and the FastAPI/Vite browser interface with collapsible tool-call cards, a workspace file sidebar, and a live token counter. Both are demonstration surfaces only; all benchmark runs execute the non-interactive loop directly.

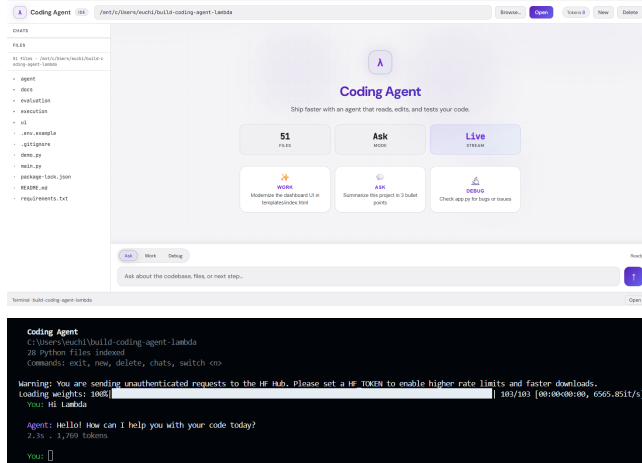


Figure 6: The two interaction surfaces over the same agent core: the FastAPI/Vite browser interface (top) and the terminal chat interface (bottom).

C Reproducibility Checklist

- (1) Record the repository commit, model endpoint/checkpoint, decoding parameters, prompt version, agent configuration, maximum iterations, and token cap.
- (2) Generate one prediction JSON record per SWE-bench instance. Persist after each task and skip completed IDs when resuming.
- (3) Preserve the raw trajectory, terminal reason, token counters, and final git diff; distinguish empty patches from evaluator failures.
- (4) Evaluate predictions with the official SWE-bench Docker harness. Do not substitute generation-environment tests for the official verdict.
- (5) Report resolved tasks, non-empty patches, patch precision, and tokens. Keep single-attempt and best-of- k results separate.
- (6) For the configuration ablation, hold model, tasks, budgets, and evaluator constant. For the model ablation, hold harness, tasks, budgets, and evaluator constant.

D Result Arithmetic

The derived percentages in the paper can be recomputed directly:

$$\text{resolve rate} = \frac{\text{resolved}}{500}, \quad \text{patch rate} = \frac{\text{patches}}{500}, \quad \text{precision} = \frac{\text{resolved}}{\text{patches}}.$$

For Run 2, for example, $185/500 = 37.0\%$, $403/500 = 80.6\%$, and $185/403 = 45.9\%$. The Run 1 to Run 2 token reduction is $(168 - 88)/168 = 47.6\%$. The ceiling-recovery ratios in Section 8.2 are $37.0/52.0 = 71.2\%$ and $34.6/73.4 = 47.1\%$. Minor display differences can result from rounding aggregate token counts to millions.

E Use of AI-Assisted Tools

AI-assisted tools were used during development for code-oriented experimentation and for editorial refinement of project artifacts. The authors remain responsible for the implementation, experiment design, source verification, reported measurements, interpretation, and final text. Project-specific numerical results are reported from team evaluation records and were cross-checked against the final presentation and source configuration; external factual claims are cited.